

TechConnect — AWS Environment Runbook

Step-by-step recreation guide for the production AWS deployment (ECS Express Mode, RDS, CloudFront). Every section gives the AWS Console path first, then the equivalent AWS CLI commands.

Account ID	440427555121
Region	eu-west-1
Application	TechConnect (backend: Spring Boot / Java 21, frontend: Angular)
Last verified	September 2026

Naming note: resources created early in this project used the pre-rename product name `techdispatch`; the app itself was later renamed to `techconnect`. Existing resources that already exist under the old name (the RDS database/role, the S3 bucket, the ECR repository, the SSM parameter path) should be reused as-is rather than recreated under a new name — renaming an S3 bucket or ECR repository isn't possible in place. Anything genuinely new should use `techconnect` - * naming going forward.

Build order (each step depends on the ones before it):

- 1 RDS PostgreSQL database
- 2 SSM Parameter Store secrets
- 3 IAM roles for ECS
- 4 ECR repository + Docker image
- 5 ECS Express Gateway service
- 6 Security group wiring (RDS ↔ ECS)
- 7 S3 bucket for the frontend build
- 8 CloudFront distribution
- 9 Post-deploy config update (app base URL)
- 10 GitHub OIDC + CI/CD deploy role (optional)
- 11 Cost budget + SNS + Lambda alert
- 12 Verification checklist

1. RDS PostgreSQL Database

A small, single-AZ Postgres instance in a private subnet — the only thing in this environment that holds real, hard-to-reproduce data, so it's the one resource in this guide you generally **don't** want to recreate casually.

Console steps

- 1 RDS console → **Databases** → **Create database**.
- 2 Engine: **PostgreSQL**. Templates: **Free tier** (or Dev/Test if outside the free tier window).
- 3 DB instance identifier: `techconnect-db`. Instance class: `db.t4g.micro` or `db.t3.micro` (free-tier eligible).
- 4 Master username: `techconnect`. Master password: generate one and save it — it becomes the value you store in SSM in the next section.
- 5 Storage: 20 GB gp2 (within the free allowance).
- 6 Connectivity: choose your VPC, **Public access: No**, create a new or existing security group (e.g. `techconnect-rds-sg` — you'll add its actual inbound rule in step 6, once the ECS service's security group exists).
- 7 Additional configuration → Initial database name: `techconnect`.
- 8 Create database. Wait for status **Available**, then note its **Endpoint** (Connectivity & security tab) — you'll need it for the ECS task definition's `POSTGRES_HOST` env var.

CLI alternative

```
aws rds create-db-instance \
  --db-instance-identifier techconnect-db \
  --db-instance-class db.t4g.micro \
  --engine postgres \
  --master-username techconnect \
  --master-user-password "<choose-a-strong-password>" \
  --allocated-storage 20 \
  --db-name techconnect \
  --vpc-security-group-ids <rds-sg-id> \
  --no-publicly-accessible \
  --region eu-west-1

# Wait for it to become available, then fetch the endpoint:
aws rds describe-db-instances --db-instance-identifier techconnect-db \
  --query "DBInstances[0].Endpoint.Address" --output text --region eu-west-1
```

2. SSM Parameter Store — Secrets

Four SecureString parameters the ECS task pulls in at container start, instead of baking secrets into the task definition or the Docker image.

Console steps

- 1 Systems Manager console → **Parameter Store** → **Create parameter**, four times, one per secret below.
- 2 For each: Tier **Standard**, Type **SecureString**, KMS key **alias/aws/ssm** (the default AWS-managed key is fine — no need for a customer-managed key).

/techdispatch/prod/db-password	-> the RDS master password from section 1
/techdispatch/prod/jwt-secret	-> openssl rand -base64 48 (generate fresh, never reuse a local d
/techdispatch/prod/mail-username	-> your SES SMTP username
/techdispatch/prod/mail-password	-> your SES SMTP password

These keep the pre-rename /techdispatch/prod/* path because that's what already exists and what the IAM policy in the next section is scoped to. There's no functional reason to rename the path — only the product-facing name changed, not this internal identifier.

CLI alternative

```
aws ssm put-parameter --name "/techdispatch/prod/db-password" --type SecureString \
  --value "<rds-master-password>" --region eu-west-1

aws ssm put-parameter --name "/techdispatch/prod/jwt-secret" --type SecureString \
  --value "$(openssl rand -base64 48)" --region eu-west-1

aws ssm put-parameter --name "/techdispatch/prod/mail-username" --type SecureString \
  --value "<ses-smtp-username>" --region eu-west-1

aws ssm put-parameter --name "/techdispatch/prod/mail-password" --type SecureString \
  --value "<ses-smtp-password>" --region eu-west-1
```

3. IAM Roles for ECS

Two roles ECS Express Mode needs: an execution role (lets the container agent pull the image, read secrets, write logs) and an infrastructure role (lets the Express Mode control plane manage the ALB/target group on your behalf).

Console steps

- 1 IAM console → **Roles** → **Create role** → Trusted entity: **AWS service** → Use case: **Elastic Container Service** → **Elastic Container Service Task**.
- 2 Name it `ecsTaskExecutionRole`. Attach managed policy `AmazonECSTaskExecutionRolePolicy`.
- 3 On the same role, add an inline policy named `techdispatch-ssm-read` (JSON below) so it can read the four parameters from section 2.
- 4 Add a second inline policy for KMS — `SecureString` parameters need `kms:Decrypt` in addition to `ssm:GetParameters`, or the task will fail with `AccessDenied` on startup even though the SSM permission looks correct.
- 5 Create a second role, same trusted-entity flow, named `ecsInfrastructureRoleForExpressServices`, and attach the managed policy `AmazonECSInfrastructureRoleforExpressGatewayServices`.
- 6 Double-check the **Trust relationships** tab on both roles shows `ecs-tasks.amazonaws.com` (execution role) / `ecs.amazonaws.com` (infrastructure role) as the trusted principal — a role created through a different console flow can end up with no trust policy at all, which fails with "cannot be assumed" the first time anything tries to use it.

SSM read policy (attach to `ecsTaskExecutionRole`)

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": "ssm:GetParameters",
    "Resource": "arn:aws:ssm:eu-west-1:440427555121:parameter/techdispatch/prod/*"
  }]
}
```

KMS decrypt policy (attach to `ecsTaskExecutionRole`)

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": "kms:Decrypt",
    "Resource": "arn:aws:kms:eu-west-1:440427555121:key/*",
    "Condition": {
      "StringEquals": { "kms:ViaService": "ssm.eu-west-1.amazonaws.com" }
    }
  }]
}
```

CLI alternative

```
aws iam create-role --role-name ecsTaskExecutionRole \
```

```
--assume-role-policy-document '{
  "Version": "2012-10-17",
  "Statement": [{"Effect": "Allow", "Principal": {"Service": "ecs-tasks.amazonaws.com"}, "Action": "s
}'}

aws iam attach-role-policy --role-name ecsTaskExecutionRole \
--policy-arn arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy

aws iam put-role-policy --role-name ecsTaskExecutionRole \
--policy-name techdispatch-ssm-read --policy-document file://ssm-read-policy.json

aws iam put-role-policy --role-name ecsTaskExecutionRole \
--policy-name techdispatch-kms-decrypt --policy-document file://kms-decrypt-policy.json

aws iam create-role --role-name ecsInfrastructureRoleForExpressServices \
--assume-role-policy-document '{
  "Version": "2012-10-17",
  "Statement": [{"Effect": "Allow", "Principal": {"Service": "ecs.amazonaws.com"}, "Action": "sts:Ass
}'}

aws iam attach-role-policy --role-name ecsInfrastructureRoleForExpressServices \
--policy-arn arn:aws:iam::aws:policy/service-role/AmazonECSInfrastructureRoleforExpressGatewayService
```

4. ECR Repository & Docker Image

Console steps

- 1 ECR console → **Repositories** → **Create repository** → name techdispatch-backend (reuse the existing repo if it's still there — don't create a second one under a different name, or every reference to it elsewhere in this guide needs updating).
- 2 Click into the repository → **View push commands** for the exact login/tag/push sequence for your OS.
- 3 From the backend/ directory of the repo, build and push the image (see CLI block — the console's own "push commands" panel gives you the same thing pre-filled with your account ID).

CLI alternative

```
cd backend
aws ecr get-login-password --region eu-west-1 | \
  docker login --username AWS --password-stdin 440427555121.dkr.ecr.eu-west-1.amazonaws.com

docker build -t 440427555121.dkr.ecr.eu-west-1.amazonaws.com/techdispatch-backend:latest .
docker push 440427555121.dkr.ecr.eu-west-1.amazonaws.com/techdispatch-backend:latest
```

If `aws ecr get-login-password` fails with an OAuth2/authorization-grant error, your AWS CLI credentials have expired or the active profile is misconfigured — run `aws sts get-caller-identity` first to confirm you have a working session before troubleshooting anything ECR-specific.

5. ECS Express Gateway Service

This single command provisions the Fargate task, an Application Load Balancer, a target group, and wires them together — it's the replacement for AWS App Runner, which stopped accepting new customers on 2026-04-30.

Console steps

- 1 ECS console → **Clusters** → **default** (or create a cluster first if none exists) → **Express services** tab → **Create**.
- 2 Container image: the ECR image URI from section 4. Container port: 8080.
- 3 Environment variables: see the table below.
- 4 Secrets: map each to its SSM parameter from section 2 (use "Value from" — the parameter's **name or ARN**, never the parameter's decrypted value itself; pasting the actual secret value here instead of a reference is a real mistake that's easy to make and causes AccessDenied errors that look like an IAM problem but aren't).
- 5 Health check path: /api/actuator/health.
- 6 Execution role: ecsTaskExecutionRole. Infrastructure role: ecsInfrastructureRoleForExpressServices.
- 7 Networking: choose the same VPC as the RDS instance from section 1 (a mismatch here is a common, hard-to-diagnose failure — ECS Express Mode can silently pick a different default VPC than your database lives in, and security groups can't reference each other across VPCs).
- 8 Create, and wait for the deployment to reach steady state. Note the **Application URL** (<https://<random>.ecs.eu-west-1.on.aws>) — this is the value you'll point CloudFront's second origin at in section 8, and it changes every time the service is deleted and recreated.

Environment variables

POSTGRES_HOST	<RDS endpoint from section 1>
POSTGRES_PORT	5432
POSTGRES_DB	techconnect
POSTGRES_USER	techconnect
MAIL_HOST	email-smtp.eu-west-1.amazonaws.com
MAIL_PORT	587
TECHCONNECT_MAIL_FROM	<a verified SES sender address>
TECHCONNECT_BOOTSTRAP_ADMIN_EMAIL	<the first admin's email>
SPRING_PROFILES_ACTIVE	prod
TECHCONNECT_APP_BASE_URL	<filled in after section 8 — see section 9>

MAIL_HOST region matters: this must match whatever region you actually verified your SES sending identity in — a mismatched region here produces a `java.net.UnknownHostException` at runtime, not an obvious config error.

Mail health check: Spring Boot Actuator auto-registers a health indicator that actively probes SMTP connectivity. If SES/mail isn't reachable, `/api/actuator/health` reports 503 even though the app itself is completely fine — which fails the ALB's health check and blocks every deployment. Add `management.health.mail.enabled=false` to `application-prod.properties` before building the image in section 4, since mail delivery is designed to be best-effort in this app (a failed send is logged, not fatal).

CLI alternative

```
aws ecs create-express-gateway-service \
  --service-name techdispatch-backend \
  --execution-role-arn arn:aws:iam::440427555121:role/ecsTaskExecutionRole \
  --infrastructure-role-arn arn:aws:iam::440427555121:role/ecsInfrastructureRoleForExpressServices \
  --health-check-path "/api/actuator/health" \
  --cpu 256 --memory 512 \
  --primary-container '{
    "image": "440427555121.dkr.ecr.eu-west-1.amazonaws.com/techdispatch-backend:latest",
    "containerPort": 8080,
    "environment": [
      {"name": "POSTGRES_HOST", "value": "<rds-endpoint>"},
      {"name": "POSTGRES_PORT", "value": "5432"},
      {"name": "POSTGRES_DB", "value": "techconnect"},
      {"name": "POSTGRES_USER", "value": "techconnect"},
      {"name": "MAIL_HOST", "value": "email-smtp.eu-west-1.amazonaws.com"},
      {"name": "MAIL_PORT", "value": "587"},
      {"name": "TECHCONNECT_MAIL_FROM", "value": "<verified-ses-sender>"},
      {"name": "TECHCONNECT_BOOTSTRAP_ADMIN_EMAIL", "value": "<admin-email>"},
      {"name": "SPRING_PROFILES_ACTIVE", "value": "prod"},
      {"name": "TECHCONNECT_APP_BASE_URL", "value": "placeholder-filled-in-after-section-8"}
    ],
    "secrets": [
      {"name": "POSTGRES_PASSWORD", "valueFrom": "arn:aws:ssm:eu-west-1:440427555121:parameter/techdispatch-postgres-password"},
      {"name": "TECHCONNECT_JWT_SECRET", "valueFrom": "arn:aws:ssm:eu-west-1:440427555121:parameter/techdispatch-jwt-secret"},
      {"name": "MAIL_USERNAME", "valueFrom": "arn:aws:ssm:eu-west-1:440427555121:parameter/techdispatch-mail-username"},
      {"name": "MAIL_PASSWORD", "valueFrom": "arn:aws:ssm:eu-west-1:440427555121:parameter/techdispatch-mail-password"}
    ]
  }' \
  --network-configuration '{"awsVpcConfiguration":{"subnets":["<rds-vpc-private-subnet-ids>"]}}' \
  --monitor-resources
```

Note the returned serviceArn and Application URL from the output.

6. Security Group Wiring (RDS ↔ ECS)

RDS sits in a private subnet and accepts nothing by default. The ECS task needs an explicit inbound rule on the RDS security group — sourced from the ECS task's own security group, not a CIDR block.

Console steps

- 1 ECS console → your Express service → **Configuration and networking** (or the **Networking** tab) → note the **Security group** ID attached to the running task (not the load balancer's own security group — check the task's actual network interface if more than one security group is listed, since Express Mode often creates one for the gateway/ALB and a separate one for the task itself).
- 2 EC2 console → **Security Groups** → the RDS instance's security group → **Edit inbound rules** → **Add rule**: Type **PostgreSQL**, Port **5432**, Source → search for and select the ECS task's security group ID (a value starting **sg-**, never a rule ID starting **sg-** — picking a rule ID by mistake from the autocomplete produces a "CIDR block, a security group ID or a prefix list has to be specified" error).
- 3 Save rules.

If saving produces "You have specified two resources that belong to different networks," the ECS service and the RDS instance are in different VPCs — security group references only work within the same VPC. The fix is deleting and recreating the ECS Express service pointed at RDS's own VPC/subnets (section 5), not adjusting this rule.

CLI alternative

```
aws ec2 authorize-security-group-ingress \
  --group-id <rds-security-group-id> \
  --protocol tcp --port 5432 \
  --source-group <ecs-task-security-group-id> \
  --region eu-west-1
```

7. S3 Bucket for the Frontend Build

Console steps

- 1 S3 console → **Create bucket** → name it something globally unique, e.g. techdispatch-frontend-<suffix> (reuse the existing bucket if it's still there).
- 2 Block all public access — CloudFront reaches it via Origin Access Control, not a public bucket policy, so this should stay fully blocked.
- 3 Build and upload the Angular app:

```
cd frontend
npm ci
npm run build -- --configuration production
aws s3 sync dist/frontend/browser s3://techdispatch-frontend-<suffix> --delete --region eu-west-1
```

CLI alternative (bucket creation)

```
aws s3 mb s3://techdispatch-frontend-<suffix> --region eu-west-1

aws s3api put-public-access-block \
  --bucket techdispatch-frontend-<suffix> \
  --public-access-block-configuration \
    BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
```

8. CloudFront Distribution

One distribution, two origins: S3 for the static Angular build, and the ECS Express service for `/api/*`. This is what makes the browser see everything as one origin, so there's no CORS configuration needed anywhere in the app.

Console steps

- 1 CloudFront console → **Origin access** → **Create control setting** → name it `techconnect-oac`.
- 2 **Distributions** → **Create distribution**.
- 3 **Origin 1**: choose the S3 bucket from section 7 from the dropdown, attach the origin access control just created (accept the console's offer to update the bucket policy for you).
- 4 Default cache behavior: **Viewer protocol policy** → **Redirect HTTP to HTTPS** (not "HTTP and HTTPS"), cache policy **CachingOptimized**.
- 5 Settings: default root object `index.html`.
- 6 Create the distribution, then go back in and add the second origin and behavior (CloudFront's create wizard only lets you fully configure one origin cleanly at a time; adding the rest after is completely normal and doesn't require recreating anything).
- 7 **Origins** tab → **Create origin** → type the ECS Express service's Application URL hostname directly into the origin domain field as free text — it won't appear in the autocomplete dropdown (that list only surfaces resource types CloudFront recognizes natively, like S3 buckets or load balancers — a custom domain like this is still fully valid, just not suggested). Protocol: **HTTPS only**, port 443.
- 8 **Do not** point this origin at the raw Application Load Balancer's own `.elb.amazonaws.com` hostname even if you can find it — that ALB typically has no valid HTTPS listener of its own; TLS termination happens at the Express service's managed `.on.aws` domain, which is what you must use here.
- 9 **Behaviors** tab → **Create behavior** → path pattern `/api/*` → origin = the ECS origin just added → cache policy **CachingDisabled** → origin request policy **AllViewer** → confirm this behavior's precedence is **0** (ahead of the default (`*`) behavior at precedence 1) so it's actually evaluated first.
- 10 **Error pages** tab → **Create custom error response**, twice: HTTP error code **403** → Customize response → response page path `/index.html` → HTTP response code **200**. Repeat for error code **404**. This is what makes refreshing the page on any Angular route (not just `/`) work instead of 403/404-ing, since those paths don't exist as real files in S3.

Watch out for: the 403/404 → 200 override above applies distribution-wide, to any origin's error response — not just S3's. If the ECS origin itself ever returns a genuine 403 or 404 (for example, from Spring Security), this same rule will mask it behind a 200 response containing the Angular app's HTML instead of the real error, which is confusing to debug. If API calls seem to silently fail, check the raw response body, not just the status code.

Missing the `/api/*` behavior entirely produces a very similar-looking symptom: every API call quietly returns the Angular `index.html` with a 200 status instead of hitting the backend at all, because the request falls through to the default S3 behavior and then gets caught by the same error-page override above.

CLI alternative

CloudFront's CLI interface for a multi-origin, multi-behavior distribution is a single large JSON document rather than incremental commands, which makes the console meaningfully faster for this specific resource. If you do want the CLI form:

```
aws cloudfront create-origin-access-control \
  --origin-access-control-config Name=techconnect-oac,SigningProtocol=sigv4,SigningBehavior=always,OriginsAllowedToSign=AllOrigins

# Then build a full distribution-config.json (two origins, two cache behaviors, two custom
# error responses) and run:
aws cloudfront create-distribution --distribution-config file://distribution-config.json

# To modify an existing distribution instead:
aws cloudfront get-distribution-config --id <distribution-id> > current-config.json
# edit the ETag-stamped config, then:
aws cloudfront update-distribution --id <distribution-id> \
  --distribution-config file://updated-config.json --if-match <etag-from-get>
```

9. Post-Deploy Config Update

Once the CloudFront distribution exists, you have its domain — feed that back into the backend so activation/reset links point at the right place.

Console steps

- 1 CloudFront console → your distribution → note the **Distribution domain name** (dxxxxxxxxxxxxx.cloudfront.net).
- 2 ECS console → your Express service → **Update service** → edit the TECHCONNECT_APP_BASE_URL environment variable to https://dxxxxxxxxxxxxx.cloudfront.net → deploy.

CLI alternative

```
aws ecs update-express-gateway-service \
  --service-arn <service-arn-from-section-5> \
  --primary-container '{"environment":[{"name":"TECHCONNECT_APP_BASE_URL","value":"https://dxxxxxxxxxxxxx"}
```

10. GitHub OIDC + CI/CD Deploy Role (optional)

Lets GitHub Actions push new images and redeploy without a long-lived AWS access key ever existing as a GitHub secret. Skip this section until the manual flow above is confirmed working end-to-end at least once.

Console steps

- 1 IAM console → **Identity providers** → **Add provider** → Provider type **OpenID Connect** → Provider URL `https://token.actions.githubusercontent.com` → Audience `sts.amazonaws.com`.
- 2 **Roles** → **Create role** → Trusted entity type **Web identity** → select the provider just created → Audience `sts.amazonaws.com` → GitHub organization/repository: your actual repo (e.g. `your-org/your-repo`).
- 3 Name it `techconnect-gha-deploy`. Attach an inline policy scoped to exactly: ECR push to the one repo, S3 sync to the one bucket, CloudFront invalidation on the one distribution, and `ecs:UpdateExpressGatewayService` on the one service — not broad account-wide permissions.
- 4 In GitHub: repo **Settings** → **Secrets and variables** → **Actions** → add secret `AWS_DEPLOY_ROLE_ARN`, and variables `AWS_REGION`, `ECR_REPOSITORY_NAME`, `EXPRESS_SERVICE_ARN`, `S3_BUCKET_NAME`, `CLOUDFRONT_DISTRIBUTION_ID`.

CLI alternative

```
aws iam create-open-id-connect-provider \
  --url https://token.actions.githubusercontent.com \
  --client-id-list sts.amazonaws.com \
  --thumbprint-list 6938fd4d98bab03faadb97b34396831e3780aea1

aws iam create-role --role-name techconnect-gha-deploy \
  --assume-role-policy-document file://trust-policy.json

aws iam put-role-policy --role-name techconnect-gha-deploy \
  --policy-name techconnect-deploy-perms --policy-document file://permissions-policy.json
```

`trust-policy.json` needs a `token.actions.githubusercontent.com:sub` condition scoped to `repo:<org>/<repo>:*` — without it, any GitHub Actions workflow anywhere could assume this role.

11. Cost Budget + SNS + Lambda Alert

A plain AWS Budgets email notification can't carry custom instructions — only a fixed AWS-worded template. To get an actual custom message (what's running, what it costs, exact commands to shut it down) you need Budget → SNS topic → Lambda → SES send.

Console steps

- 1 SNS console → **Topics** → **Create topic** → Standard → name `techconnect-budget-alert`.
- 2 Lambda console → **Create function** → Author from scratch → name `techconnect-budget-alert`, runtime Python 3.13.
- 3 Execution role: create a new role (or reuse an existing one) trusted by `lambda.amazonaws.com` — check its **Trust relationships** tab explicitly if you created the role through a different console flow, since a role created for another purpose can end up with no Lambda trust statement at all, which fails with "cannot be assumed by Lambda" the moment you try to attach it.
- 4 Attach `AWSLambdaBasicExecutionRole` (CloudWatch Logs) and an inline policy granting `ses:SendEmail`.
- 5 Paste in the custom send-email code (see `lambda_function.py` alongside this document), deploy.
- 6 SNS console → the topic → **Create subscription** → protocol **AWS Lambda** → select the function (this also grants SNS permission to invoke it automatically).
- 7 Billing console → **Budgets** → your budget → **Edit** → **Notifications** → add a threshold at **95%** of actual cost → recipient type **Amazon SNS topic** → select the topic above. Keep a separate, simpler 80% plain-email notification if you want an earlier heads-up too.

SES sandbox mode requires **both** the sender and every recipient address to be individually verified identities, or the Lambda's send call fails silently — check `/aws/lambda/techconnect-budget-alert` in CloudWatch Logs if no email arrives.

AWS Budgets' own "actual spend" figure lags the Billing dashboard / Cost Explorer by up to 24 hours. A budget showing \$0 right after creating or editing it, while the dashboard shows real spend, is normal and not a sign of misconfiguration.

CLI alternative

```
aws sns create-topic --name techconnect-budget-alert --region eu-west-1

zip function.zip lambda_function.py
aws lambda create-function --function-name techconnect-budget-alert \
  --runtime python3.13 --handler lambda_function.handler \
  --role arn:aws:iam::440427555121:role/techconnect-budget-alert-role \
  --zip-file fileb://function.zip --region eu-west-1

aws sns subscribe --topic-arn arn:aws:sns:eu-west-1:440427555121:techconnect-budget-alert \
  --protocol lambda \
  --notification-endpoint arn:aws:lambda:eu-west-1:440427555121:function:techconnect-budget-alert

aws lambda add-permission --function-name techconnect-budget-alert \
  --statement-id sns-invoke --action lambda:InvokeFunction \
  --principal sns.amazonaws.com \
  --source-arn arn:aws:sns:eu-west-1:440427555121:techconnect-budget-alert

# Test without waiting for a real threshold:
```

```
aws sns publish --topic-arn arn:aws:sns:eu-west-1:440427555121:techconnect-budget-alert \  
--message "test" --region eu-west-1
```


12. Verification Checklist

- 1 `curl https://<express-service>.ecs.eu-west-1.on.aws/api/actuator/health` returns `{"status": "UP"}` — confirms the backend, DB connection, and IAM/secrets wiring are all correct, independent of CloudFront.
- 2 Open the CloudFront domain in a browser — the Angular app loads.
- 3 Submit the sign-in form (or any API-calling action) and confirm, in the browser's network tab, that the request actually reaches `/api/*` and returns real JSON — not the SPA's own `index.html` in disguise (see the note in section 8).
- 4 CloudWatch Logs for the ECS task shows the bootstrap admin's activation link (logged regardless of whether the email itself sends successfully). Open it, activate the account, log in.
- 5 `psql` into RDS directly (temporarily allow your IP on its security group, then revert) and confirm the expected tables/rows exist.

Generated for the TechConnect project — account 440427555121, region eu-west-1. Treat every password/secret value in this document as a placeholder to be generated fresh, never reused from a prior environment.